

ML Profiling Topics

Nsight Systems topics relevant to ML/DL on DGX Spark / GB10

Purpose. This handout maps Nsight Systems documentation to practical ML/DL profiling topics for GB10, with emphasis on PyTorch/vLLM workloads, CUDA kernel visibility, unified memory, Tensor Core utilization, Python overhead, containers/KFP, and post-processing reports.

GB10 context. DGX Spark uses NVIDIA Grace Blackwell architecture with integrated CPU/GPU, a 20-core Arm CPU, Blackwell GPU with 5th Generation Tensor Cores, 128 GB LPDDR5x unified memory, 273 GB/s memory bandwidth, and support for inference/deployment/fine-tuning of large AI models. [DGX Spark hardware overview](#).

Relevant NVIDIA docs

Doc / section	Use it for	Link
Nsight Systems User Guide - Focused Profiling	Limit captures to performance-critical ML phases instead of collecting entire jobs.	Focused profiling
Nsight Systems User Guide - NVTX Marking and Labeling	Annotate train/eval/inference phases and correlate CPU ranges to GPU work.	NVTX ranges
Nsight Systems User Guide - CLI Profile Options	Capture ranges, CUDA tracing, CUDA flush interval, CUDA Graph, UM fault, backtrace, and trace-scope options.	CLI options
Nsight Systems User Guide - PyTorch Profiling	Auto-annotate PyTorch forward/backward/step operations and vLLM methods.	PyTorch profiling
Nsight Systems User Guide - GPU Metrics	Sample SM activity, Tensor Active, memory bandwidth, warp occupancy, and IO throughput.	GPU Metrics
Nsight Systems Analysis Guide - Statistical Reports	Understand nsys stats reports such as cuda_gpu_kern_sum, cuda_gpu_trace, cuda_api_sum, and launch/exec summaries.	Analysis reports
DGX Spark User Guide - Hardware Overview	Ground GB10 analysis in actual CPU/GPU/memory/Tensor Core hardware characteristics.	DGX Spark hardware

1. Focused profiling windows

Why it matters: Long ML jobs generate too much trace data. Profile only the phase that answers the performance question: warmup, forward, backward, optimizer, eval, checkpoint, vLLM prefill, or vLLM decode.

Relevant docs: [Focused Profiling](#); [NVTX Marking and Labeling](#); [capture range options](#).

GB10 / ML focus:

- Add NVTX ranges around train/eval/inference sub-phases.
- Prefer `--capture-range=nvtx` and `--capture-range-end=stop-shutdown` for controlled windows.
- Avoid whole KFP/pipeline captures except when debugging launch/container behavior.

Example command:

```
nsys profile --trace=cuda,nvtx,cudnn,cublas,osrt \
--capture-range=nvtx \
--capture-range-end=stop-shutdown \
-o report_ml_window \
python -u train_or_infer.py
```

2. CUDA kernel visibility and CUDA summaries

Why it matters: Core sanity check: did Nsight actually retain GPU kernel activity? For your run-035 type issue, `cuda_api_sum` without `cuda_gpu_kern_sum` points to capture/reporting/process/trace-volume issues, not necessarily a GB10 limitation.

Relevant docs: [default nsys stats reports](#); [CUDA GPU Kernel Summary](#); [CUDA GPU Trace](#).

GB10 / ML focus:

- Check `cuda_gpu_kern_sum`, `cuda_gpu_trace`, `cuda_api_sum`, `cuda_gpu_mem_time_sum`, `cuda_gpu_mem_size_sum`.
- Use `cuda_kern_exec_sum` to split API launch time, queue time, and kernel time.
- Compare standalone matmul/GEMM vs full KFP pipeline to isolate environment vs workload behavior.

Example command:

```
nsys stats --report cuda_gpu_kern_sum \
--report cuda_gpu_trace \
--report cuda_api_sum \
--format column report.nsys-rep
```

3. PyTorch and vLLM profiling

Why it matters: Nsight can automatically annotate common PyTorch operations and, with `functions-trace`, vLLM methods such as `AsyncLLM.generate()`, `Worker.execute_model()`, and `Worker.sample_tokens()`.

Relevant docs: [PyTorch Profiling](#).

GB10 / ML focus:

- Use `functions-trace` for lower-overhead operation ranges.
- Use `functions-trace-shapes` only for short windows because shape metadata increases volume.
- Combine with `autograd-nvtx` for training phases.

Example command:

```
nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx \
--pytorch=functions-trace-shapes,autograd-nvtx \
--python-sampling=true \
--python-backtrace=cuda \
-o torch_train_profile python -u train.py
```

4. GB10 GPU metrics: SM, Tensor Core, memory bandwidth

Why it matters: On Blackwell, you want to know whether the GPU is idle, full, Tensor Core active, memory-bandwidth limited, warp-limited, or IO-limited.

Relevant docs: [GPU Metrics overview](#); [DGX Spark performance specifications](#).

GB10 / ML focus:

- Track SM activity, Tensor Active, warp occupancy, instruction issue rate, unassigned warp slots, PCIe/NVLink/GPU memory bandwidth.
- Requires supported GPU and elevated permissions for performance counters.
- Use lower-frequency sampling if diagnostics report buffer overflow.

Example command:

```
sudo nsys profile --trace=cuda,nvtx,cudnn,cublas,osrt \  
--gpu-metrics-devices=all \  
--gpu-metrics-frequency=10000 \  
-o gb10_gpu_metrics python -u benchmark.py
```

5. Unified memory and page-fault behavior

Why it matters: GB10 unified memory lets large models fit, but CPU/GPU access patterns, page faults, KV-cache growth, and memory locality can still dominate.

Relevant docs: [CUDA UM CPU/GPU page-fault switches](#); [DGX Spark unified memory](#).

GB10 / ML focus:

- Use UM page-fault tracing only on short repos; NVIDIA warns it can cause significant overhead.
- Correlate page faults with model load, checkpointing, dataloader access, KV-cache growth, and eval bursts.
- Watch memory bandwidth and Tensor Active together: low Tensor Active plus memory pressure can indicate memory-bound behavior.

Example command:

```
nsys profile --trace=cuda,nvtx,osrt \  
--cuda-um-cpu-page-faults=true \  
--cuda-um-gpu-page-faults=true \  
--capture-range=nvtx \  
--capture-range-end=stop-shutdown \  
-o gb10_um_short python -u memory_repro.py
```

6. Python, CPU, dataloader, tokenizer, and GIL overhead

Why it matters: ML workloads often starve the GPU from Python, tokenization, dataloading, logging, MLflow, checkpoint serialization, or CPU scheduling.

Relevant docs: [Python Backtrace Sampling](#); [Python Functions Trace](#); [Python GIL Tracing](#).

GB10 / ML focus:

- Use Python sampling to see where Python spends wall time.

- Use Python backtrace on CUDA triggers to connect Python call sites to CUDA work.
- Use GIL tracing when many Python threads exist but GPU is idle.

Example command:

```
nsys profile --trace=cuda,nvtx,osrt,python-gil \
--python-sampling=true \
--python-sampling-frequency=200 \
-o python_overhead python -u train.py
```

7. CUDA Graphs, cuBLAS, cuDNN, CUTLASS, and attention kernels

Why it matters: Training and inference performance often hinges on whether work is dominated by matmul/attention Tensor Core kernels or by small elementwise/reduction/sampling kernels.

Relevant docs: [CUDA Graph trace option](#); [CLI trace options](#).

GB10 / ML focus:

- Use `--trace=cuda,cudnn,cublas,nvtx,osrt` as the baseline ML trace set.
- For vLLM/PyTorch CUDA Graph use, prefer `--cuda-graph-trace=graph`; node-level tracing is heavier.
- Use kernel names and Tensor Active to separate GEMM/attention from launch-heavy or memory-heavy sections.

Example command:

```
nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx \
--cuda-graph-trace=graph \
-o graph_aware_profile python -u benchmark_vllm.py
```

8. Synchronization and stream behavior

Why it matters: Unexpected synchronization often explains idle gaps: `cudaDeviceSynchronize`, `cudaStreamSynchronize`, cuda events, `memcpy` synchronization, or metric/logging calls that force GPU completion.

Relevant docs: [CUDA Event trace option](#); [CUDA kernel launch and execution summary](#).

GB10 / ML focus:

- Use `cuda_kern_exec_sum` to inspect API time, queue time, and kernel time.
- Queue time is not always bad; no queue time everywhere may indicate an idle GPU.
- Enable CUDA event trace selectively because it may increase overhead and create false dependency-like effects.

Example command:

```
nsys stats --report cuda_kern_exec_sum \
--report cuda_kern_exec_trace \
report.nsys-rep
```

9. Container, Kubernetes, and KFP profiling

Why it matters: KFP and containerized runs add process-tree, permissions, seccomp/perf_event_open, output-path, child-process, and sidecar issues that standalone shell tests do not exercise.

Relevant docs: [Collecting Data Within a Container](#); [Profiling Kubernetes services with Nsight Operator](#).

GB10 / ML focus:

- Prefer the Nsight CLI inside the workload container or a Kubernetes-aware injection approach.
- Check that perf_event_open and CUDA tracing are permitted.
- Confirm nsys version, CUDA_VISIBLE_DEVICES, output mount, process tree, and report finalization path inside the pod.

Example command:

```
PYTHONUNBUFFERED=1 nsys profile --trace=cuda,nvtx,cudnn,cublas,osrt \  
--cuda-trace-scope=process-tree \  
-o /outputs/kfp_component_profile \  
python -u component_entrypoint.py
```

10. Long-run trace volume and buffer behavior

Why it matters: Nsight Systems documentation warns that, in general, it does not support runs over 5 minutes. Long captures can fill storage, inflate reports, and lose analytical usefulness.

Relevant docs: [duration and long-run warning](#); [CUDA flush interval option](#).

GB10 / ML focus:

- For collections over 30 seconds, the docs recommend a 10-second CUDA flush interval.
- Focused NVTX windows are safer than whole-job captures.
- Report absence or missing sections after long runs should be treated as a profiler/capture health signal.

Example command:

```
nsys profile --trace=cuda,nvtx,cudnn,cublas,osrt \  
--cuda-flush-interval=10000 \  
--capture-range=nvtx \  
--capture-range-end=stop-shutdown \  
-o focused_long_ml python -u train.py
```

11. Post-processing and automated report checks

Why it matters: Your run-monitoring skills should treat nsys stats output as machine-checkable artifacts: which report sections exist, which are empty, and how CUDA/GPU/Python/OS metrics change across runs.

Relevant docs: [Post-Collection Analysis Guide](#); [nsys stats multiple report example](#).

GB10 / ML focus:

- Export CSV for `cuda_gpu_trace`, `cuda_gpu_kern_sum`, `cuda_api_sum`, memory summaries, NVTX summaries, and OS runtime summaries.
- Add pass/fail checks for missing `cuda_gpu_kern_sum`, no kernels, suspiciously tiny kernel time, or empty NVTX ranges.
- Use SQLite export for custom comparisons across KFP runs.

Example command:

```
nsys stats --report cuda_gpu_trace \
--report cuda_gpu_kern_sum \
--report cuda_api_sum \
--format csv,column \
--output .- report.nsys-rep
```

Practical checklist for DGX Spark / GB10 ML profiling

- **Start with a short CUDA sanity test:** Confirm `cuda_gpu_kern_sum` exists before blaming GB10, Blackwell, CUPTI, PyTorch, or KFP.
- **Use NVTX capture windows:** Profile one train/eval/inference phase at a time; avoid 47-minute all-in traces unless debugging collection behavior.
- **Add PyTorch/vLLM annotations:** Enable `functions-trace` for inference and add `autograd-nvtx` for training windows.
- **Turn on GPU metrics for hardware questions:** Use SM Active, Tensor Active, memory bandwidth, and warp occupancy to decide whether you are compute-, memory-, or launch-limited.
- **Treat UM tracing as a scalpel:** Enable page-fault tracing only for short repros because overhead can distort real workload behavior.
- **Separate container/KFP from workload behavior:** Run the same Python workload both standalone and under KFP/container profiling to isolate environment effects.
- **Automate report validation:** Fail runs that produce `cuda_api_sum` but no CUDA GPU kernels, empty NVTX ranges, or no report sections after long captures.

Source notes

Nsight Systems version checked: NVIDIA Nsight Systems User Guide v2026.3 and Post-Collection Analysis Guide v2026.3, plus DGX Spark User Guide hardware overview last updated June 1, 2026.

Important caveat: Nsight feature availability can depend on target architecture, driver version, container permissions, and elevated access to GPU performance counters. Always verify commands on the target DGX Spark host.